Experiment-1

```
# 8 Puzzle Problem using BFS

from collections import deque

# 0 represents the empty block

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)


# Print the puzzle
def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()


# Generate possible moves
def get_moves(state):

    moves = []

    # Find the empty block
    empty = state.index(0)

    # Positions that the empty block can move to
    possible_moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }

    for position in possible_moves[empty]:

        new_state = list(state)

        # Swap empty block and tile
        new_state[empty], new_state[position] = \
            new_state[position], new_state[empty]

        moves.append(tuple(new_state))

    return moves


# BFS
queue = deque()

queue.append((start, [start]))

visited = set()
visited.add(start)

while queue:

    state, path = queue.popleft()

    # Check goal
    if state == goal:

        print("Solution Found!")
        print("Number of moves:", len(path) - 1)
```

```
        print("Number of moves:", len(path) - 1)
        print()

        # Display solution
        for i in range(len(path)):
            print("Step", i)
            print_puzzle(path[i])

        break

    # Find next possible states
    for next_state in get_moves(state):

        if next_state not in visited:

            visited.add(next_state)

            queue.append(
                (next_state, path + [next_state])
            )

else:
    print("No solution")
```

```
Solution Found!
Number of moves: 2

Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0
```